МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 1

по дисциплине «Математические основы баз данных»

по теме: «Проектирование и создание структуры базы данных»

Выполнил студент группы:
БВТ2505
Соронина Арина Андреевна
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цель и задачи работы:

Цель: спроектировать и реализовать структуру базы данных согласно описанию предметной области.

Задачи:

1. Проведите анализ предметной области и сформулируйте перечень бизнес-правил
2. Постройте ER-диаграмму, включающую не менее десяти сущностей. Схема должна содержать хотя бы одну связь «многие-ко-многим» через ассоциативную таблицу и хотя бы одну иерархическую связь (самоссылку).
3. Приведите схему к третьей нормальной форме. На примере одной таблицы покажите аномалию до нормализации и поясните, как она устранена.
4. Определите для каждой таблицы первичный ключ; обоснуйте выбор суррогатных или естественных ключей.
5. Реализуйте схему в PostgreSQL средствами DDL: создайте таблицы с корректными типами, первичными и внешними ключами, ограничениями NOT NULL, UNIQUE, CHECK, DEFAULT и правилами ON DELETE / ON UPDATE.
6. Составьте словарь данных: для каждой таблицы перечислите столбцы, их типы, ограничения и назначение.
7. Оформите все команды единым SQL-скриптом, повторное выполнение которого воссоздаёт схему с нуля.

Выполнение

1. Анализ предметной области и формулировка перечня бизнес-правил.

Предметная область – сеть кинотеатров: продажа билетов на киносеансы. Клиент бронирует места на сеанс конкретного фильма в конкретном зале и оплачивает бронирование. Также присутствуют справочные данные (жанры, прокатчики, актёры, конфигурация залов).

Основные бизнес-правила:

1. Каталог и справочники
    a. У фильма есть ровно один жанр; один жанр может быть присвоен многим фильмам.
    b. У фильма не более одного прокатчика; прокатчик может быть не указан.
    c. Название жанра уникально в рамках каталога.
    d. В актёрском составе фильма может быть несколько актёров; актёр может сниматься в нескольких фильмах.
    e. Для каждой пары «фильм – актёр» фиксируется ровно одна роль.
2. Залы и места
    a. Каждое место принадлежит ровно одному залу.
    b. Место однозначно определяется парой «ряд + номер» и не может повторяться в пределах одного зала.
    c. Вместимость зала – положительное число.
    d. Тип зала – один из фиксированного набора форматов показа (2D, 3D, IMAX).

3. Сеансы
    a. Сеанс показывает ровно один фильм в ровно одном зале.
    b. Один зал не может проводить два сеанса, начинающихся в одно и то же время.
    c. Цена сеанса – положительное число.
4. Клиенты, бронирования и билеты
    a. Бронирование принадлежит ровно одному клиенту.
    b. Бронирование должно содержать хотя бы один билет – пустых бронирований не существует.
    c. Билет относится ровно к одному сеансу и ровно к одному месту.
    d. Одно место на конкретный сеанс может быть продано только один раз.
    e. Статус бронирования принимает одно из фиксированных значений: pending, confirmed, cancelled.
    f. Телефон клиента обязателен и уникален; email не обязателен, но если указан – уникален и должен быть в корректном формате.
5. Оплата
    a. На бронирование приходится не более одного платежа, и наоборот – платёж относится ровно к одному бронированию (связь 1:1).
    b. Сумма платежа – положительное число; способ оплаты – один из фиксированного набора (card, cash, online).
6. Правила, которые не сводятся к ограничению схемы
    a. Нельзя продать билет на сеанс, который уже начался.

b. Отменить бронирование можно только до начала сеанса.

c. Сумма платежа должна совпадать с суммой цен всех билетов бронирования – это межтабличная сверка, а не ограничение одной колонки.

d. Возврат оформляется отдельной операцией, а не прямым удалением платежа – иначе теряется финансовая история.

2. *Построение ER-диаграммы.*

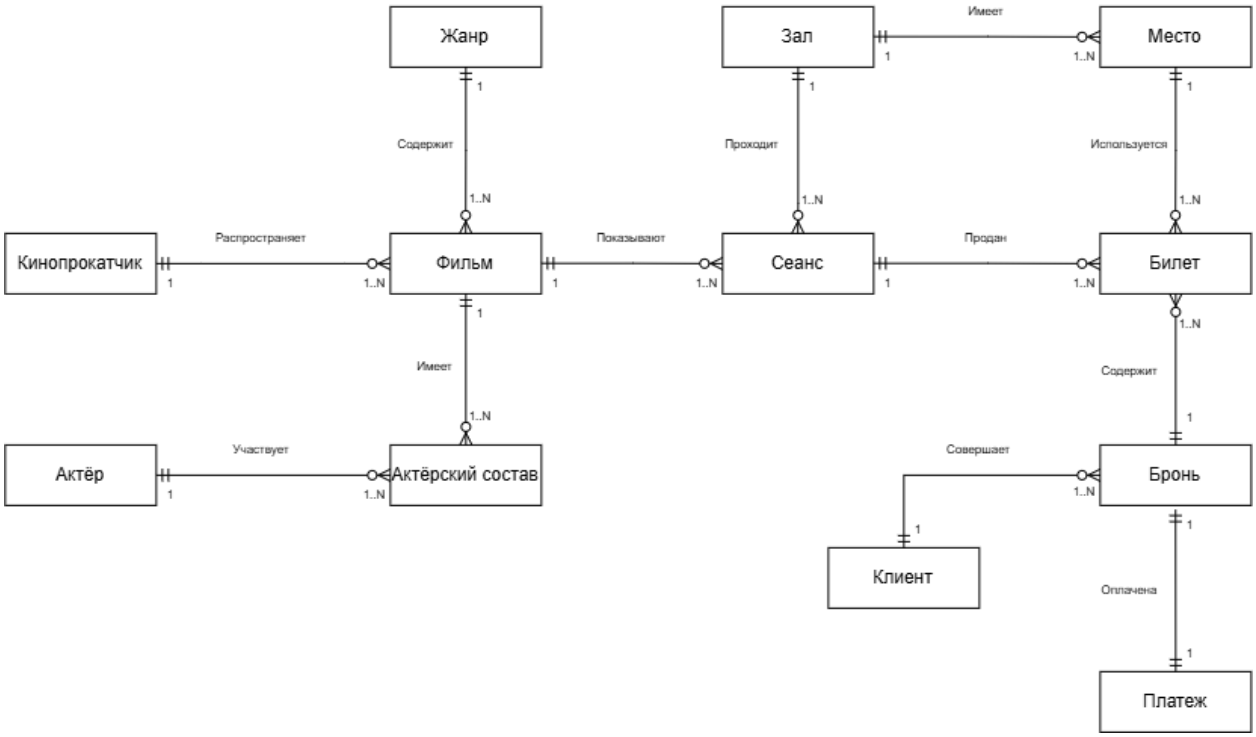


Рисунок 1. Концептуальная ER-модель предметной области кинотеатра.

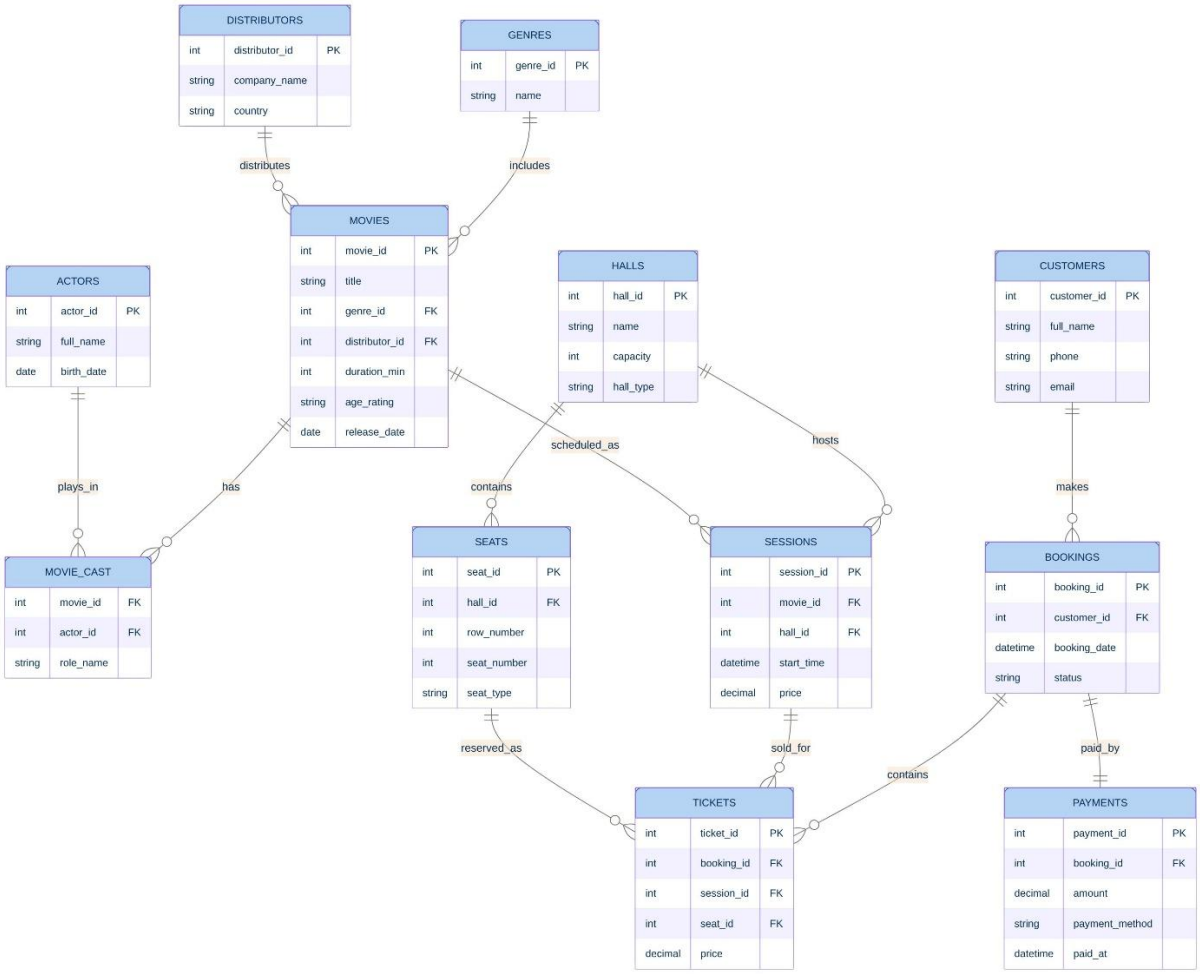


Рисунок 2. Логическая схема базы данных (диаграмма «сущность – связь»)

12 связей – 10 обычных «один-ко-многим», одна «один-к-одному» (Бронь–Платёж) и одна «многие-ко-многим» (Фильм–Актёр), реализованная через промежуточную таблицу Актёрский состав – единственный способ выразить M:N в реляционной модели.

3. *Приведение схемы к третьей нормальной форме и демонстрация аномалии.*

Прежде чем показывать аномалию, разберём саму итоговую схему кинотеатра по всем трём нормальным формам.

a. 1НФ – выполняется. Во всех атрибутах (title, phone, start_time и т. д.) хранятся атомарные значения; повторяющихся групп и списков в одной ячейке нет.
b. 2НФ – выполняется. Единственная таблица с составным ключом – Movie_Cast (movie_id + actor_id). Атрибут role_name зависит от пары «фильм + актёр» целиком, а не от одной части ключа – частичной зависимости нет.
c. 3НФ – выполняется. Описательные атрибуты (название жанра, страна прокатчика и т. п.) вынесены в отдельные справочники (Genres, Distributors), а в «дочерних» таблицах остаются только внешние ключи, а не сами названия. Транзитивных зависимостей нет.

Итоговая схема уже соответствует 3NF. Чтобы показать, зачем это нужно, ниже разобран пример, который нарушает 3NF, если спроектировать таблицу «в лоб» – без вынесения справочников в отдельные сущности.

Допустим, на первом шаге проектирования данные о жанре и прокатчике просто вписали прямо в таблицу фильмов, не вынося их в отдельные сущности:

movie_id (PK)	**title**	**genre_id**	**genre_name**	**distributor_id**	**company_name**	**country**
1	Дюна	10	Фантастика	100	Warner Bros	США
2	Интерстеллар	10	Фантастика	101	Paramount	США
3	Левиафан	20	Драма	102	Нон-Стоп	Россия

Первичный ключ таблицы – movie_id. Он однозначно определяет genre_id и distributor_id – это нормально. Но genre_name зависит не от movie_id, а от genre_id (аналогично company_name и country зависят от distributor_id, а не от movie_id напрямую). Возникает цепочка зависимости:

movie_id → genre_id → genre_name

Это транзитивная зависимость: неключевой атрибут (genre_name) зависит от другого неключевого атрибута (genre_id), а не напрямую от первичного ключа. Именно такие зависимости запрещает третья нормальная форма.

Три аномалии на этих данных:

1. Аномалия вставки – нельзя занести в базу новый жанр, например «Документальный», пока не появится хотя бы один фильм этого жанра – жанр существует только как текст внутри строки Movies, отдельной записи для него нет.
2. Аномалия обновления – если жанр «Фантастика» решили переименовать в «Научная фантастика», нужно обновить обе строки – id 1 и id 2. Если обновить только одну, в базе окажутся два разных названия одного и того же жанра – данные несогласуются.
3. Аномалия удаления – если удалить фильм «Левиафан» (id 3) — единственный в базе с жанром «Драма» и единственный от прокатчика «Нон-Стоп» – вместе со строкой бесследно исчезнет и сама информация о том, что жанр «Драма» и прокатчик «Нон-Стоп» вообще существуют.

После нормализации устраняется проблема с помощью декомпозиции, которая уже заложена в итоговой ER-диаграмме: жанр и прокатчик вынесены в отдельные сущности, а Movies хранит только ссылки на них.

Genres	
genre_id (PK)	name
10	Фантастика
20	Драма

Distributors		
distributor_id (PK)	company_name	country
100	Warner Bros	США
101	Paramount	США
102	Нон-Стоп	Россия

Movies			
movie_id (PK)	title	genre_id (FK)	distributor_id (FK)
1	Дюна	10	100
2	Интерстеллар	10	101
3	Левиафан	20	102

Теперь genre_name зависит напрямую от genre_id – ключа таблицы Genres, а не «проходит транзитом» через Movies. Транзитивная зависимость исчезла, и вместе с ней – все три аномалии:

1. Вставка – жанр «Документальный» можно добавить в Genres отдельной строкой в любой момент, независимо от того, есть ли уже такой фильм.
2. Обновление – название жанра хранится ровно в одном месте – строка genre_id = 10 в Genres. Переименование – это одно обновление, а не поиск всех фильмов этого жанра.
3. Удаление – удаление фильма «Левиафан» из Movies никак не затрагивает строки «Драма» в Genres и «Нон-Стоп» в Distributors – они существуют независимо и связаны с фильмами только через внешний ключ.

4. Определение первичных, суррогатных и естественных ключей.

Общий принцип, применённый в схеме: там, где естественный атрибут нестабилен, не гарантированно уникален в будущем или содержит персональные данные, выбран суррогатный PK. Там, где сама связь (пара внешних ключей) уже однозначно и стабильно определяет строку, оставлен составной естественный ключ или переиспользован родительский ключ.

Таблица	**Первичный ключ**	**Тип**	**Обоснование**
Genres	genre_id	Суррогатный	Естественный кандидат – name, но названия жанров переименовывают; PK на тексте каскадно ломает все внешние ссылки при переименовании.
Distributors	distributor_id	Суррогатный	company_name не гарантированно уникально (тёзки в разных странах) и может меняться при ребрендинге или слиянии компаний.
Movies	movie_id	Суррогатный	Естественного уникального атрибута нет: title не уникален (ремейки, сиквелы), даже title + release_date не спасает.
Actors	actor_id	Суррогатный	full_name не уникален (однофамильцы); full_name + birth_date почти работает, но дата рождения часто неизвестна.
Movie_Cast	movie_id, actor_id (составной)	Естественный	Пара внешних ключей сама однозначно и осмысленно определяет строку – «этот актёр в этом фильме». Отдельный cast_id только усложнил бы схему.
Halls	hall_id	Суррогатный	Кандидат name («Зал 1») работает для одного здания, но при расширении сети кинотеатров номера залов начнут повторяться между площадками.
Seats	seat_id	Суррогатный*	Есть надёжный кандидат (hall_id, row_number, seat_number) – место определяется этой тройкой. Суррогат удобнее для FK в Tickets, но тройку стоит защитить UNIQUE.
Sessions	session_id	Суррогатный*	Кандидат (hall_id, start_time) уникален: зал не может показывать два фильма одновременно. Суррогат удобнее для FK, но UNIQUE(hall_id, start_time) стоит сохранить.
Customers	customer_id	Суррогатный	Email/телефон – персональные данные: меняются, могут временно совпадать в семье, нежелательно тиражировать их как ключ в Bookings.
Bookings	booking_id	Суррогатный	У бронирования нет естественного идентифицирующего признака – это транзакция/событие, а не объект реального мира с собственным «именем».
Tickets	ticket_id	Суррогатный*	Кандидат (session_id, seat_id) критичен для правила «одно место на сеанс продаётся только

Таблица	Первичный ключ	Тип	Обоснование
			раз» – его защищают UNIQUE даже при суррогатном PK.
Payments	booking_id (shared key)	Естественный	Связь Bookings – Payments строго 1:1, поэтому booking_id одновременно PK и FK – это гарантирует 1:1 на уровне схемы, а не «по факту».

** – суррогатный PK выбран для удобства внешних ключей, но естественная составная комбинация существует и должна быть закреплена отдельным ограничением UNIQUE.*

5. Словарь данных.

genres			
Столбец	Тип	Ограничения	Назначение
genre_id	INTEGER (IDENTITY)	PK	Уникальный идентификатор жанра
name	VARCHAR(100)	NOT NULL, UNIQUE	Название жанра

distributors			
Столбец	Тип	Ограничения	Назначение
distributor_id	INTEGER (IDENTITY)	PK	Идентификатор прокатчика/студии
company_name	VARCHAR(150)	NOT NULL	Название компании-прокатчика
country	VARCHAR(100)	NOT NULL	Страна регистрации компании

actors			
Столбец	Тип	Ограничения	Назначение
actor_id	INTEGER (IDENTITY)	PK	Идентификатор актёра
full_name	VARCHAR(150)	NOT NULL	ФИО актёра
birth_date	DATE	CHECK (≤ текущей даты), NULL допустим	Дата рождения; не всегда известна

halls			
Столбец	Тип	Ограничения	Назначение
hall_id	INTEGER (IDENTITY)	PK	Идентификатор зала
name	VARCHAR(50)	NOT NULL, UNIQUE	Название/номер зала
capacity	INTEGER	NOT NULL, CHECK (> 0)	Вместимость зала
hall_type	VARCHAR(10)	NOT NULL, DEFAULT '2D', CHECK IN ('2D','3D','IMAX')	Формат показа в зале

customres			
Столбец	Тип	Ограничения	Назначение
customer_id	INTEGER (IDENTITY)	PK	Идентификатор клиента
full_name	VARCHAR(150)	NOT NULL	ФИО клиента
phone	VARCHAR(20)	NOT NULL, UNIQUE	Телефон — основной контакт
email	VARCHAR(150)	UNIQUE, CHECK (формат email), NULL допустим	Email; регистрация возможна и без него

movies			
Столбец	Тип	Ограничения	Назначение
movie_id	INTEGER (IDENTITY)	PK	Идентификатор фильма
title	VARCHAR(200)	NOT NULL	Название фильма
genre_id	INTEGER	NOT NULL, FK → genres, ON DELETE RESTRICT	Жанр фильма
distributor_id	INTEGER	FK → distributors, ON DELETE SET NULL, NULL допустим	Прокатчик; может быть не указан
duration_min	INTEGER	NOT NULL, CHECK (> 0)	Длительность в минутах
age_rating	VARCHAR(5)	NOT NULL, CHECK IN ('0+'…'18+')	Возрастной рейтинг
release_date	DATE	NOT NULL	Дата выхода фильма

movie_cast			
Столбец	Тип	Ограничения	Назначение
movie_id	INTEGER	PK (составной), FK → movies, ON DELETE CASCADE	Фильм, в котором задействован актёр
actor_id	INTEGER	PK (составной), FK → actors, ON DELETE CASCADE	Актёр, снявшийся в фильме
role_name	VARCHAR(150)	NOT NULL	Название роли

seats			
Столбец	Тип	Ограничения	Назначение
seat_id	INTEGER (IDENTITY)	PK	Идентификатор места
hall_id	INTEGER	NOT NULL, FK → halls, ON DELETE CASCADE	Зал, к которому принадлежит место
row_number	INTEGER	NOT NULL, CHECK (> 0)	Номер ряда
seat_number	INTEGER	NOT NULL, CHECK (> 0)	Номер места в ряду
seat_type	VARCHAR(10)	NOT NULL, DEFAULT 'standard', CHECK IN ('standard','vip')	Категория места
—	—	UNIQUE (hall_id, row_number, seat_number)	Место в зале не может повторяться

sessions			
Столбец	Тип	Ограничения	Назначение
session_id	INTEGER (IDENTITY)	PK	Идентификатор сеанса
movie_id	INTEGER	NOT NULL, FK → movies, ON DELETE RESTRICT	Какой фильм показывают
hall_id	INTEGER	NOT NULL, FK → halls, ON DELETE RESTRICT	В каком зале идёт сеанс
start_time	TIMESTAMP	NOT NULL	Дата и время начала сеанса
price	NUMERIC(8,2)	NOT NULL, CHECK (> 0)	Базовая цена билета на сеанс
—	—	UNIQUE (hall_id, start_time)	Зал не показывает два сеанса одновременно

bookings			
Столбец	Тип	Ограничения	Назначение
booking_id	INTEGER (IDENTITY)	PK	Идентификатор бронирования
customer_id	INTEGER	NOT NULL, FK → customers, ON DELETE RESTRICT	Клиент, оформивший бронь
booking_date	TIMESTAMP	NOT NULL, DEFAULT now()	Дата и время создания брони
status	VARCHAR(20)	NOT NULL, DEFAULT 'pending', CHECK IN (...)	Текущий статус брони

tickets			
Столбец	Тип	Ограничения	Назначение
ticket_id	INTEGER (IDENTITY)	PK	Идентификатор билета
booking_id	INTEGER	NOT NULL, FK → bookings, ON DELETE CASCADE	Бронирование, в которое входит билет
session_id	INTEGER	NOT NULL, FK → sessions, ON DELETE RESTRICT	Сеанс, на который куплен билет
seat_id	INTEGER	NOT NULL, FK → seats, ON DELETE RESTRICT	Место, купленное по билету
price	NUMERIC(8,2)	NOT NULL, CHECK (> 0)	Фактическая цена билета
—	—	UNIQUE (session_id, seat_id)	Одно место на сеанс продаётся только раз

payments			
Столбец	Тип	Ограничения	Назначение
booking_id	INTEGER	PK, FK → bookings, ON DELETE RESTRICT (shared key)	Бронирование, к которому относится оплата (1:1)
amount	NUMERIC(10,2)	NOT NULL, CHECK (> 0)	Сумма платежа
payment_method	VARCHAR(10)	NOT NULL, CHECK IN ('card','cash','online')	Способ оплаты
paid_at	TIMESTAMP	NOT NULL, DEFAULT now()	Дата и время платежа

6. Единый SQL-скрипт.

```
CREATE TABLE genres (genre_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name VARCHAR(100) NOT NULL UNIQUE)

CREATE TABLE distributors (distributor_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, company_name VARCHAR(150) NOT NULL, country VARCHAR(100) NOT NULL);

CREATE TABLE actors (actor_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, full_name VARCHAR(150) NOT NULL, birth_date DATE, CONSTRAINT chk_actors_birth_date CHECK (birth_date is NULL OR birth_date <= CURRENT_DATE));

CREATE TABLE halls (hall_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name VARCHAR(50) NOT NULL UNIQUE, capacity INTEGER NOT NULL CHECK (capacity > 0), hall_type VARCHAR(10) NOT NULL DEFAULT '20' CHECK (hall_type IN ('2D', '3D', 'IMAX')));

CREATE TABLE customers (customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, full_name VARCHAR(150) NOT NULL, phone VARCHAR(20) NOT NULL UNIQUE, email VARCHAR(150) UNIQUE, CONSTRAINT chk_customers_email CHECK (email IS NULL OR email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'));

CREATE TABLE movies (movie_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, title VARCHAR(200) NOT NULL, genre_id INTEGER NOT NULL REFERENCES genres(genre_id) ON DELETE RESTRICT ON UPDATE CASCADE, distributor_id INTEGER REFERENCES distributors(distributor_id) ON DELETE SET NULL ON UPDATE CASCADE, duration_min INTEGER NOT NULL CHECK (duration_min > 0), age_rating VARCHAR(5) NOT NULL CHECK (age_rating IN ('0+', '6+', '12+', '16+', '18+')), release_date DATE NOT NULL);

CREATE TABLE movie_cast (movie_id INTEGER NOT NULL REFERENCES movies(movie_id) ON DELETE CASCADE ON UPDATE CASCADE, actor_id INTEGER NOT NULL REFERENCES actors(actor_id) ON DELETE CASCADE ON UPDATE CASCADE, role_name VARCHAR(150) NOT NULL, PRIMARY KEY(movie_id, actor_id));

CREATE TABLE seats (seat_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, hall_id INTEGER NOT NULL REFERENCES halls(hall_id) ON DELETE CASCADE ON UPDATE CASCADE, row_number INTEGER NOT NULL CHECK (row_number > 0), seat_number INTEGER NOT NULL CHECK (seat_number > 0), seat_type VARCHAR(10) NOT NULL DEFAULT 'standart' CHECK (seat_type IN ('standart', 'vip')), UNIQUE (hall_id, row_number, seat_number));

CREATE TABLE sessions (session_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, movie_id INTEGER NOT NULL REFERENCES movies(movie_id) ON DELETE RESTRICT ON UPDATE CASCADE, hall_id INTEGER NOT NULL REFERENCES halls(hall_id) ON DELETE RESTRICT ON UPDATE CASCADE, start_time TIMESTAMP NOT NULL, price NUMERIC(8,2) NOT NULL CHECK (price > 0), UNIQUE (hall_id, start_time));

CREATE TABLE bookings (booking_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, customer_id INTEGER NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT ON UPDATE CASCADE, booking_date TIMESTAMP NOT NULL DEFAULT now(), status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed', 'cancelled')));

CREATE TABLE tickets (ticket_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, booking_id INTEGER NOT NULL REFERENCES bookings(booking_id) ON DELETE CASCADE ON UPDATE CASCADE, session_id INTEGER NOT NULL REFERENCES sessions(session_id) ON DELETE RESTRICT ON UPDATE CASCADE, seat_id INTEGER NOT NULL REFERENCES seats(seat_id) ON DELETE RESTRICT ON UPDATE CASCADE, price NUMERIC(8,2) NOT NULL CHECK (price > 0), UNIQUE (session_id, seat_id));

CREATE TABLE payments (booking_id INTEGER PRIMARY KEY REFERENCES bookings(booking_id) ON DELETE RESTRICT ON UPDATE CASCADE, amount NUMERIC(10,2) NOT NULL CHECK (amount > 0), payment_method VARCHAR(10) NOT NULL CHECK (payment_method IN ('card', 'cash', 'online')), paid_at TIMESTAMP NOT
```

```
NULL DEFAULT now());CREATE TABLE genres (genre_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name VARCHAR(100) NOT NULL UNIQUE)

CREATE TABLE distributors (distributor_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, company_name VARCHAR(150) NOT NULL, country VARCHAR(100) NOT NULL);

CREATE TABLE actors (actor_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, full_name VARCHAR(150) NOT NULL, birth_date DATE, CONSTRAINT chk_actors_birth_date CHECK (birth_date is NULL OR birth_date <= CURRENT_DATE));

CREATE TABLE halls (hall_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name VARCHAR(50) NOT NULL UNIQUE, capacity INTEGER NOT NULL CHECK (capacity > 0), hall_type VARCHAR(10) NOT NULL DEFAULT '20' CHECK (hall_type IN ('2D', '3D', 'IMAX')));

CREATE TABLE customers (customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, full_name VARCHAR(150) NOT NULL, phone VARCHAR(20) NOT NULL UNIQUE, email VARCHAR(150) UNIQUE, CONSTRAINT chk_customers_email CHECK (email IS NULL OR email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'));

CREATE TABLE movies (movie_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, title VARCHAR(200) NOT NULL, genre_id INTEGER NOT NULL REFERENCES genres(genre_id) ON DELETE RESTRICT ON UPDATE CASCADE, distributor_id INTEGER REFERENCES distributors(distributor_id) ON DELETE SET NULL ON UPDATE CASCADE, duration_min INTEGER NOT NULL CHECK (duration_min > 0), age_rating VARCHAR(5) NOT NULL CHECK (age_rating IN ('0+', '6+', '12+', '16+', '18+')), release_date DATE NOT NULL);

CREATE TABLE movie_cast (movie_id INTEGER NOT NULL REFERENCES movies(movie_id) ON DELETE CASCADE ON UPDATE CASCADE, actor_id INTEGER NOT NULL REFERENCES actors(actor_id) ON DELETE CASCADE ON UPDATE CASCADE, role_name VARCHAR(150) NOT NULL, PRIMARY KEY(movie_id, actor_id));

CREATE TABLE seats (seat_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, hall_id INTEGER NOT NULL REFERENCES halls(hall_id) ON DELETE CASCADE ON UPDATE CASCADE, row_number INTEGER NOT NULL CHECK (row_number > 0), seat_number INTEGER NOT NULL CHECK (seat_number > 0), seat_type VARCHAR(10) NOT NULL DEFAULT 'standart' CHECK (seat_type IN ('standart', 'vip')), UNIQUE (hall_id, row_number, seat_number));

CREATE TABLE sessions (session_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, movie_id INTEGER NOT NULL REFERENCES movies(movie_id) ON DELETE RESTRICT ON UPDATE CASCADE, hall_id INTEGER NOT NULL REFERENCES halls(hall_id) ON DELETE RESTRICT ON UPDATE CASCADE, start_time TIMESTAMP NOT NULL, price NUMERIC(8,2) NOT NULL CHECK (price > 0), UNIQUE (hall_id, start_time));

CREATE TABLE bookings (booking_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, customer_id INTEGER NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT ON UPDATE CASCADE, booking_date TIMESTAMP NOT NULL DEFAULT now(), status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed', 'cancelled')));

CREATE TABLE tickets (ticket_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, booking_id INTEGER NOT NULL REFERENCES bookings(booking_id) ON DELETE CASCADE ON UPDATE CASCADE, session_id INTEGER NOT NULL REFERENCES sessions(session_id) ON DELETE RESTRICT ON UPDATE CASCADE, seat_id INTEGER NOT NULL REFERENCES seats(seat_id) ON DELETE RESTRICT ON UPDATE CASCADE, price NUMERIC(8,2) NOT NULL CHECK (price > 0), UNIQUE (session_id, seat_id));

CREATE TABLE payments (booking_id INTEGER PRIMARY KEY REFERENCES bookings(booking_id) ON DELETE RESTRICT ON UPDATE CASCADE, amount NUMERIC(10,2) NOT NULL CHECK (amount > 0), payment_method VARCHAR(10) NOT NULL CHECK (payment_method IN ('card', 'cash', 'online')), paid_at TIMESTAMP NOT NULL DEFAULT now());
```

Вывод

Предметная область доведена от бизнес-правил до рабочей БД: 12 сущностей приведены к 3NF (устранены аномалии вставки/обновления/удаления), для каждой таблицы обоснованно выбран тип первичного ключа, схема реализована в PostgreSQL с полным набором ограничений (PK/FK, NOT NULL, UNIQUE, CHECK, DEFAULT, ON DELETE/UPDATE) и задокументирована словарём данных.

Большинство бизнес-правил (кардинальности связей, обязательность и диапазоны значений) гарантируется самой структурой БД. Правила, зависящие от времени или требующие сверки нескольких строк (например, «нельзя продать билет на начавшийся сеанс»), вынесены на уровень приложения — SQL-ограничениями они не выражаются.